# M. แผ่นสูตรเร็วท้ายเล่ม (อ่าน 3 นาทีก่อนเข้าห้อง)

### 📖 รู้จักส่วนนี้ก่อนอ่าน

**ความหมาย:** สรุปสูตรคำนวณทุกบท + ตัวอย่างไล่ขั้นตอนฉบับเต็ม 7 แบบ (TAT/WT, แปลง address, EAT, นับ fault, Segmentation, Banker's, inode/rwx)

**ความสำคัญกับข้อสอบ:** เป็น index ระหว่างทำข้อสอบ — ตันหัวข้อไหนกลับมาเปิดหน้านี้ก่อน แล้วค่อยไล่ไปส่วนเต็มที่อยู่หน้าก่อน ๆ

**ใช้จริงในคอมพิวเตอร์:** สูตรพวกนี้คือสิ่งที่ OS คำนวณจริงทุกมิลลิวินาที (แปลง address ทุกครั้งที่เข้าถึง RAM) — เข้าใจสูตร = เข้าใจการทำงานเครื่อง

## ⏱️ CPU Scheduling

| | |
|---|---|
| **TAT** | Completion − Arrival |
| **WT** | TAT − Burst (หา TAT ก่อนเสมอ!) |
| **RR** | รอไม่เกิน (n−1)q · q ใหญ่=FCFS |
| **เท่ากัน** | เลือกมาถึงก่อน (FCFS tiebreak) |

## 🧮 Paging

| | |
|---|---|
| **offset** | $\log_2$(page size) บิต |
| **page#** | บิต address − บิต offset |
| **เพจ** | ceil(ขนาด ÷ page size) — เริ่มนับ 0 |
| **PA** | (frame × pageSize) + offset |
| **EAT** | 2 + ε − α (ตย. ε=0.1, α=0.8 → 1.3µs) |
| **2-level** | inner = ceil(เพจ ÷ 1024) × 4KB + 4KB |

## 💾 Page Replacement (7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1 / 3 frames)

| | | |
|---|---|---|
| **FIFO** | 15 | เข้าก่อนออกก่อน · มี Belady (3f=9, 4f=10) |
| **OPT** | 9 | มองอนาคต ใช้จริงไม่ได้ |
| **LRU** | 12 | มองอดีต ใช้จริงได้ |
| **EAT(p)** | (1−p)×mat + p×service | |
| **128×128** | row นอก = 128 · col นอก = 16,384 | |

## 🏦 Banker's · 📁 File

| | |
|---|---|
| **Need** | Max − Allocation |
| **ให้ได้ไหม** | Req ≤ Need และ ≤ Avail แล้ว safety ใหม่ |
| **seg** | d < Limit → PA = Base + d (≥ = trap) |
| **fit** | First=หลุมแรก Best=เหลือน้อยสุด Worst=ใหญ่สุด |
| **inode** | entries = block÷ptr (4K/4B=1024) · 48KB+4MB+4GB+4TB |
| **rwx** | r=4 w=2 x=1 · 761 = rwx rw- --x |

## 📝 ตัวอย่างการคำนวณฉบับเต็ม (ไล่ขั้นตอนทีละบรรทัด — ตัวเลขชุดเดียวกับตัวอย่างในส่วน F/H และที่ซ้อมใน L)

### ⏱️ ตย.1 · TAT/WT — SJF Non-preemptive

| Proc | Arrival | Burst |
|---|---|---|
| P1 | 0 | 8 |
| P2 | 2 | 4 |
| P3 | 1 | 1 |

ขั้น 1: t=0 คิวมีแค่ P1 → รัน 0→8 (non-preemptive ห้ามแย่ง)
ขั้น 2: t=8 คิวมี P2(4), P3(1) → เลือก burst สั้นสุด = P3 รัน 8→9
ขั้น 3: P2 รันต่อ 9→13 · Gantt: `P1 0–8 | P3 8–9 | P2 9–13`

| Proc | TAT = จบ − มาถึง | WT = TAT − Burst |
|---|---|---|
| P1 | 8−0 = **8** | 8−8 = **0** |
| P2 | 13−2 = **11** | 11−4 = **7** |
| P3 | 9−1 = **8** | 8−1 = **7** |

### 🧮 ตย.2 · แปลง Address (Paging)

โปรแกรม 16 B เพจละ 4 B (offset 2 บิต) · Page Table: 0→f5, 1→f6, 2→f1, 3→f2 — หา PA ของ logical **6** และ **2**

| | |
|---|---|
| **ขั้น 1** | page# = logical ÷ ขนาดเพจ (ปัดเศษทิ้ง) · offset = logical mod ขนาดเพจ |
| **ลอจิก 6** | 6 ÷ 4 = **page 1** เศษ 2 → offset 2 → PT: page 1 = frame 6 → PA = 6×4 + 2 = **26** |
| **ลอจิก 2** | 2 ÷ 4 = **page 0** เศษ 2 → offset 2 → PT: page 0 = frame 5 → PA = 5×4 + 2 = **22** |

### ⚡ ตย.3 · EAT (TLB)

| | |
|---|---|
| **โจทย์** | RAM 1 µs · TLB ε = 0.1 µs · hit ratio α = 0.8 |
| **ขั้น 1 hit** | α×(ε + 1) = 0.8×1.1 = 0.88 (หาเจอ → อ่าน RAM 1 ครั้ง) |
| **ขั้น 2 miss** | (1−α)×(ε + 2) = 0.2×2.1 = 0.42 (ไม่เจอ → อ่าน RAM 2 ครั้ง) |
| **ขั้น 3 รวม** | EAT = 0.88 + 0.42 = **1.3 µs** (ตรงสูตร 2 + ε − α = 2.1 − 0.8 ✓) |
| **กลับด้าน** | ε=0.25 ต้องการ EAT = 1.35 → h×1.25 + (1−h)×2.25 = 1.35 → 2.25 − h = 1.35 → **h = 0.90** |

### 📐 ตย.4 · Segmentation

Segment Table (จากส่วน L12.2): seg 1 → base 6300, limit 400 · seg 2 → base 4300, limit 400

| | |
|---|---|
| **2 : 300** | offset 300 < limit 400 ✓ → PA = 4300 + 300 = **4600** |
| **1 : 400** | offset 400 ≥ limit 400 ✗ → **trap** (Addressing Error — offset ต้อง < limit เท่านั้น) |

## 💾 ตย.5 · นับ Page Fault — FIFO (ทีละสเต็ป)

เฟรม 3 ช่อง เริ่มมี [2, 0, 1] (2 เก่าสุด) · สตริงที่เหลือ `7 0 1 2 0 3 0 4 2 3 0`

| อ้าง | ผล (FIFO = ไล่เก่าสุด) | เฟรมหลังจบ |
|---|---|---|
| 7 | ✚ fault — แทน 2 | [7,0,1] |
| 0 | ✓ hit | [7,0,1] |
| 1 | ✓ hit | [7,0,1] |
| 2 | ✚ fault — แทน 0 | [1,7,2] |
| 0 | ✚ fault — แทน 1 | [7,2,0] |
| 3 | ✚ fault — แทน 7 | [2,0,3] |
| 0 | ✓ hit | [2,0,3] |
| 4 | ✚ fault — แทน 2 | [0,3,4] |
| 2 | ✚ fault — แทน 0 | [3,4,2] |
| 3 | ✓ hit | [3,4,2] |
| 0 | ✚ fault — แทน 3 | [4,2,0] |
| **รวม** | **FIFO = 7 faults** · สตริงเดิม OPT = 5 (มองอนาคต) · LRU = 7 (มองอดีต) | |

อาร์เรย์ 128×128 (เพจละ 1 แถว, เฟรมเดียว): วนตามแถว (row-major) = **128** faults · วนตามคอลัมน์ = **16,384** faults (ทุกช่องคนละเพจ)

## 🏦 ตย.6 · Banker's — เช็ก Safety

Available = (3,3,2) · Need = Max − Allocation → P0(7,4,3) P1(1,2,2) P2(6,0,0) P3(0,1,1) P4(4,3,1) — วนหาใคร Need ≤ Work แล้วปล่อยคืนทั้ง Allocation

| จบก่อน | เงื่อนไข | Work ใหม่ (คืนของ) |
|---|---|---|
| P1 | (1,2,2) ≤ (3,3,2) ✓ | (5,3,2) |
| P3 | (0,1,1) ≤ (5,3,2) ✓ | (7,4,3) |
| P4 | (4,3,1) ≤ (7,4,3) ✓ | (7,4,5) |
| P2 | (6,0,0) ≤ (7,4,5) ✓ | (10,4,7) |
| P0 | (7,4,3) ≤ (10,4,7) ✓ | จบครบทุกตัว |

Safe sequence = `<P1, P3, P4, P2, P0>` → ระบบ **Safe** ✓ (ถ้าวนแล้วไม่มีใครผ่านต่อ = Unsafe)

## 📁 ตย.7 · inode & rwx

| | |
|---|---|
| **ขั้น 1** | entries/บล็อก = ขนาดบล็อก ÷ ขนาด pointer = 4096 ÷ 4 = **1024** |
| **Direct 12** | 12 × 4KB = **48 KB** |
| **Single** | 1024 บล็อก × 4KB = **4 MB** |
| **Double** | 1024 × 1024 × 4KB = 1,048,576 × 4KB = **4 GB** |
| **Triple** | $1024^3$ × 4KB = **4 TB** → รวม 48KB + 4MB + 4GB + 4TB |
| **อีกแบบ** | มีเฉพาะ Double, บล็อก 2KB: 512 × 512 × 2KB = 262,144 × 2KB = **512 MB** |
| **rwx** | 761 → owner 7 = rwx · group 6 = rw- · other 1 = --x |

### 🚨 กลยุทธ์ห้องสอบ (จากรีวิวรุ่นพี่)

• ถูก/ผิด: ถูก +2 ผิด −1 ไม่ตอบ 0 — **ไม่แน่จริงอย่าตอบ** · • ทำโจทย์คำนวณ (Paging/Replacement (การแทนที่)/Scheduling) **ก่อนข้อทฤษฎี** คะแนน/นาทีสูงกว่า · • จับเวลาข้อคำนวณ ≤ 2 นาที ติดให้ข้าม · • เขียนขั้นตอนสั้น ๆ ข้างคำตอบ · • โค้ดทุกข้อ compile ผ่านหมด — ไม่มีตอบ Error!